

Documento técnico de diseño

Luis Miguel Ossa Arias

Javier Ricardo Sandoval

Jose Luis Mora Serrano

Facultad de Ingeniería, Universidad de La Sabana

Arquitectura de Software

Miguel Alfonso Varela Fonseca

22 de mayo de 2026

Abstract

Ecommify es un marketplace de e-commerce brasileño cuyo dataset público (Kaggle/Olist) comprende aproximadamente 1.45 millones de registros distribuidos en nueve archivos CSV. El equipo adoptó una Arquitectura Políglota Híbrida para gestionar la heterogeneidad inherente de los datos: PostgreSQL para los módulos transaccionales estructurados (órdenes, clientes, vendedores, pagos y reseñas), donde las garantías ACID y la integridad referencial son críticas; y MongoDB para los módulos semi-estructurados (catálogo de productos con especificaciones variables por categoría y logs de eventos). El análisis exploratorio del dataset consolidado OrdersFull reveló violaciones sistemáticas de las tres primeras formas normales: grupos de datos repetidos (1FN), dependencias parciales en claves compuestas (2FN) y dependencias transitivas como ciudad/estado dependiendo del código postal en lugar de la clave primaria del cliente (3FN). Estas violaciones motivaron una normalización hasta 3FN que derivó en nueve entidades relacionales: CUSTOMER, ORDER, ORDER_ITEM, PAYMENT, REVIEW, PRODUCT, PRODUCT_CATEGORY, SELLER y GEOLOCATION. Para el módulo transaccional en PostgreSQL se seleccionaron tipos de datos avanzados — JSONB, TEXT[], TSTZRANGE, ENUM y POINT — que reducen complejidad de consulta, mejoran eficiencia de almacenamiento y habilitan operadores especializados. La arquitectura resultante equilibra la consistencia de los datos transaccionales con la flexibilidad necesaria para gestionar un catálogo en constante evolución.

Tabla de contenidos

Lista de tablas	5
Lista de figuras.....	6
Información del Equipo y Enfoque Arquitectónico	7
Análisis de Requisitos.....	7
Contexto de Ecommify.....	7
Requisitos Funcionales.....	8
Requisitos No Funcionales.....	9
Identificación de Entidades del Dominio	10
Tabla de Volúmenes de Datos Esperados	11
Diseño Conceptual.....	12
Diagrama Entidad-Relación Completo	12
Descripción Detallada de Entidades y Atributos.....	13
CUSTOMER	13
ORDER.....	14
ORDER_ITEM.....	15
PAYMENT	16
REVIEW.....	17
PRODUCT.....	18
PRODUCT_CATEGORY.....	19
SELLER.....	20
GEOLOCATION.....	20
Relaciones y Cardinalidades	21
Restricciones de Negocio Identificadas	22
Tipos de Datos Avanzados en PostgreSQL.....	24
Rol de MongoDB en la Arquitectura Políglota Híbrida.....	25
Diseño Lógico — Módulo PostgreSQL.....	27
Esquema Relacional Normalizado (3FN)	27
Especificación Tabla por Tabla.....	27
Tabla: geolocation	27
Tabla: product_category	28
Tabla: customer	28
Tabla: seller	28

Tabla: product.....	29
Tabla: orders.....	30
Tabla: order_item	31
Tabla: payment	31
Tabla: review	32
Aplicación de Tipos de Datos Avanzados.....	32
Extensiones de PostgreSQL Utilizadas	33
Estrategia de Particionamiento	34
Triggers y Vistas Materializadas	35
Estrategia de Indexación	36
Diseño Lógico Preliminar — Módulo MongoDB	37
Criterio de Asignación: PostgreSQL vs MongoDB	37
Colecciones Principales.....	38
Esquema de Documentos (Ejemplos JSON).....	38
Colección: products	38
Colección: event_logs.....	39
Colección: user_sessions	40
Patrones de Modelado Aplicados	41
Flujo de Sincronización PostgreSQL ↔ MongoDB	42
Decisiones arquitectónicas justificadas.....	43
Matriz de decisión por entidad	43
Análisis del Teorema CAP aplicado	44
Prioridad CP (Consistencia > Disponibilidad)	44
Prioridad AP (Disponibilidad > Consistencia).....	44
Diagrama CAP	45
Trade-offs y estrategias de mitigación	45
Flujos de sincronización.....	46
Arquitectura Híbrida Propuesta	47
Flujo A — Change Data Capture (CDC).....	47
Flujo B — Event-Driven con Outbox Pattern	47
Catálogo de Eventos Principales	48
Anexos	48
Diccionario de Datos	48

Scripts SQL	48
Consultas de Ejemplo.....	48

Lista de tablas

Tabla 1. Entidades del dominio	10
Tabla 2. Volúmenes de datos esperados	11
Tabla 3. Descripción de customer.....	13
Tabla 4. Descripción de order.....	14
Tabla 5. Descripción de order_item.....	15
Tabla 6. Descripción de payment.....	16
Tabla 7. Descripción de review	17
Tabla 8. Descripción de product	18
Tabla 9. Descripción de product_category	19
Tabla 10. Descripción de seller.....	20
Tabla 11. Descripción de geolocation.....	20
Tabla 12. Relaciones y cardinalidades	21
Tabla 13. Restricciones del negocio	22
Tabla 14. Tipo de datos avanzados	24
Tabla 15. Especificación geolocation	27
Tabla 16. Especificación product_category	28
Tabla 17. Especificación customer	28
Tabla 18. Especificación seller	29
Tabla 19. Especificación product.....	29
Tabla 20. Especificación orders	30
Tabla 21. Especificación order_item	31
Tabla 22. Especificación payment	31
Tabla 23. Especificación review	32
Tabla 24. Aplicación datos avanzados	33
Tabla 25. Extensiones utilizadas	33
Tabla 26. Estrategia de indexación	36
Tabla 27. Matriz de criterio	37
Tabla 28. Patrones aplicados.....	41
Tabla 29. Matriz de decisión.....	43
Tabla 30. Prioridad CP.....	44
Tabla 31. Prioridad AP	44
Tabla 32. Trade-offs.....	45
Tabla 33. Eventos principales	48

Lista de figuras

Ilustración 1. Módulo Transaccional Ecommify (3FN).....	13
Ilustración 2. Esquema DDL de particionamiento	33
Ilustración 3. Esquema products	37
Ilustración 4. Esquema event_logs	38
Ilustración 5. Esquema user_sessions	39
Ilustración 6. Distribución CAP.....	43
Ilustración 7. Arquitectura	45

Información del Equipo y Enfoque Arquitectónico

El presente documento es elaborado por el equipo conformado por Luis Miguel Ossa Arias, Javier Ricardo Sandoval y Jose Luis Mora Serrano, estudiantes de la Maestría en Arquitectura de Software de la Universidad de La Sabana, en el marco del curso Diseño y Optimización de Bases de Datos, bajo la orientación del docente Miguel Alfonso Varela Fonseca.

El enfoque arquitectónico seleccionado es la Arquitectura Políglota Híbrida, que combina dos motores de bases de datos especializados según la naturaleza de cada tipo de dato:

- PostgreSQL para el módulo transaccional: gestiona datos estructurados con requisitos estrictos de consistencia, integridad referencial y soporte ACID. Esto incluye entidades como órdenes, clientes, pagos, reseñas y vendedores.
- MongoDB para el módulo NoSQL: gestiona datos semi-estructurados o con esquema variable, como el catálogo de productos (cuyas especificaciones difieren por categoría) y los logs de eventos operacionales.

Esta decisión se fundamenta en el principio de selección óptima de tecnología según la naturaleza del dato (polyglot persistence), evitando forzar todos los datos a un único motor y aprovechando las fortalezas de cada uno. La elección es factible en un contexto académico y empresarial: ambas tecnologías son de código abierto, ampliamente documentadas y con soporte activo de la comunidad.

Análisis de Requisitos

Contexto de Ecommify

Ecommify es un marketplace de comercio electrónico basado en el dataset público de Olist (disponible en Kaggle), correspondiente a operaciones reales de un e-commerce brasileño con presencia en múltiples estados del país. La plataforma actúa como intermediario entre vendedores independientes y compradores, gestionando el ciclo completo de vida de un pedido: desde la compra hasta la entrega y la reseña post-servicio.

El dataset comprende nueve archivos CSV con un total aproximado de 1.45 millones de registros, distribuidos en entidades como clientes, órdenes, ítems de orden, pagos, reseñas, productos, categorías de productos, vendedores y datos de geolocalización. Este volumen y diversidad de datos hacen de Ecommify un caso representativo de los desafíos de persistencia en plataformas de e-commerce modernas.

La naturaleza heterogénea del dato es el factor clave que justifica la arquitectura políglota: mientras los datos transaccionales (pedidos, pagos, relaciones cliente-vendedor) son altamente estructurados y requieren integridad referencial, el catálogo de productos presenta atributos variables según la categoría — un electrónico requiere voltaje y garantía; una prenda de vestir, talla y material —, lo que se adapta mejor a un modelo de documento (MongoDB).

Requisitos Funcionales

Del análisis del dataset y el dominio de negocio se identificaron los siguientes requisitos funcionales:

1. Registro y autenticación de clientes: el sistema debe gestionar cuentas de compra (`customer_id` como sesión) e identidad real del comprador (`customer_unique_id`), permitiendo rastrear múltiples compras de un mismo usuario.

2. Gestión del catálogo de productos: soporte para productos con categorías en portugués e inglés y atributos físicos (peso, dimensiones) variables por categoría. El catálogo es gestionado en MongoDB por su naturaleza semi-estructurada.
3. Procesamiento de pedidos con ciclo de vida completo: registro del estado de cada orden a través de los estados: created, approved, invoice, processing, shipped, delivered, con soporte para órdenes canceladas e inaccesibles.
4. Gestión de ítems por pedido: cada orden puede contener múltiples productos de distintos vendedores, con precio y valor de flete individual por ítem.
5. Procesamiento de pagos múltiples: una orden puede ser pagada con múltiples métodos simultáneos (credit_card, debit_card, boleto, voucher), con soporte para cuotas.
6. Gestión de vendedores: registro de vendedores con geolocalización, permitiendo análisis de proximidad cliente-vendedor.
7. Sistema de reseñas post-entrega: captura de puntuación Likert (1-5), título y cuerpo del comentario, con marca temporal de solicitud y respuesta.
8. Geolocalización de clientes y vendedores: resolución de coordenadas geográficas por código postal (ZIP prefix de 5 dígitos), con consolidación de múltiples coordenadas por zona.

Requisitos No Funcionales

1. Consistencia e integridad referencial: garantías ACID para el módulo transaccional en PostgreSQL. Ningún ítem de orden puede existir sin su orden padre; ningún pago sin su orden asociada.
2. Escalabilidad de datos estáticos: la tabla de geolocalización con más de 1 millón de registros debe soportar consultas de JOIN geográfico eficientes mediante índices B-Tree en zip_code_prefix.

3. Flexibilidad del catálogo: el esquema del catálogo de productos en MongoDB debe permitir agregar nuevos atributos por categoría sin requerir migraciones de esquema (ALTER TABLE).
4. Rendimiento en consultas frecuentes: índices especializados en columnas de alta cardinalidad (customer_unique_id en orders, product_id en order_items, order_purchase_timestamp para análisis temporal).
5. Normalización hasta 3FN: el módulo transaccional debe estar libre de dependencias parciales y transitivas, eliminando anomalías de inserción, actualización y borrado.
6. Tipos de datos precisos: uso de NUMERIC para montos monetarios (evitando errores de punto flotante), TIMESTAMPTZ para timestamps (soporte multi-zona horaria Brasil), CHAR para identificadores de longitud fija (hashes MD5).
7. Extensibilidad: la arquitectura debe permitir incorporar nuevos módulos (logs, análisis, recomendaciones) sin afectar el core transaccional.

Identificación de Entidades del Dominio

Del análisis de los nueve archivos CSV originales y del DataFrame consolidado OrdersFull se identificaron las siguientes entidades del dominio, cada una asociada a la violación de forma normal que motivó su separación:

Tabla 1. Entidades del dominio

Entidad	Descripción del Dominio	Registros en Dataset	Violación en OrdersFull
CUSTOMER	Cuenta de compra de un cliente. Una persona puede tener varios registros de sesión.	99,441	3FN: city y state dependían de zip_code_prefix, no de customer_unique_id (dependencia transitiva).
ORDER	Pedido con ciclo de	99,441	1FN: los datos de la orden se

	vida completo: compra -> aprobación -> envío -> entrega.		repetían en cada fila de producto asociado al pedido.
ORDER_ITEM	Ítem individual dentro de una orden. Vincula orden, producto y vendedor. Entidad débil de ORDER.	112,650	2FN: atributos como price dependen solo de order_item_id, no de la PK compuesta completa.
PAYMENT	Pago asociado a una orden. Una orden puede tener múltiples métodos de pago simultáneos.	103,886	3FN: payment_type como literal repetido generaba dependencia transitiva de dominio.
REVIEW	Reseña dejada por el cliente tras recibir su pedido. Incluye puntuación y comentario opcional.	99,224	3FN: review_score y comentarios estaban mezclados con datos de orden en OrdersFull.
PRODUCT	Producto del marketplace. Nombres anonimizados por Olist. Atributos físicos variables por categoría.	32,951	2FN/3FN: atributos físicos (peso, dimensiones) dependen solo de product_id dentro de la PK compuesta.
PRODUCT_CATEGORY	Catálogo de categorías en portugués brasileño e inglés. Clave natural estable.	71	3FN: name_english depende de category_name, no de product_id (dependencia transitiva clásica).
SELLER	Vendedor registrado en la plataforma. Geolocalizado por código postal.	3,095	3FN: city y state del vendedor dependían transitivamente de zip_code_prefix, no de seller_id.
GEOLOCATION	Coordenadas geoespaciales por código postal. Múltiples filas por ZIP en el dataset original.	1,000,163	1FN: múltiples filas por zip_code_prefix generaban grupos repetitivos de coordenadas.

Tabla de Volúmenes de Datos Esperados

La siguiente tabla consolida los volúmenes actuales del dataset y las proyecciones de crecimiento anual estimadas para una plataforma de e-commerce en expansión, junto con el motor asignado y el tipo de carga predominante:

Tabla 2. Volúmenes de datos esperados

Tabla	Registros Actuales	Crecimiento Estimado (anual)	Tipo de Carga	Motor Asignado	Particion ado
customer	99,441	+30%	OLTP intensivo	PostgreSQL	No
order	99,441	+35%	OLTP intensivo	PostgreSQL	Por fecha
order_item	112,650	+35%	OLTP / OLAP	PostgreSQL	Por fecha
payment	103,886	+35%	OLTP crítico	PostgreSQL	No
review	99,224	+30%	OLAP / escritura periódica	PostgreSQL	No
product (catalog)	32,951	+20%	Lectura intensiva	MongoDB	Por categoría
product_catego ry	71	Estable (<5% anual)	Catálogo estático	PostgreSQL	No
seller	3,095	+25%	OLTP moderado	PostgreSQL	No
geolocation	1,000,163	Estable	Referencia estática	PostgreSQL	No
logs / eventos (futuro)	N/A	Alto volumen (+50%)	Escritura masiva	MongoDB	Por fecha

Diseño Conceptual

Diagrama Entidad-Relación Completo

El siguiente diagrama Entidad-Relación (ER) representa el modelo conceptual del módulo transaccional de Ecommify normalizado hasta la Tercera Forma Normal (3FN). Cada entidad identificada en la Sección 2.4 se convierte en una tabla independiente, y las relaciones entre ellas se derivan directamente del análisis de las violaciones de forma normal presentes en el DataFrame consolidado OrdersFull.

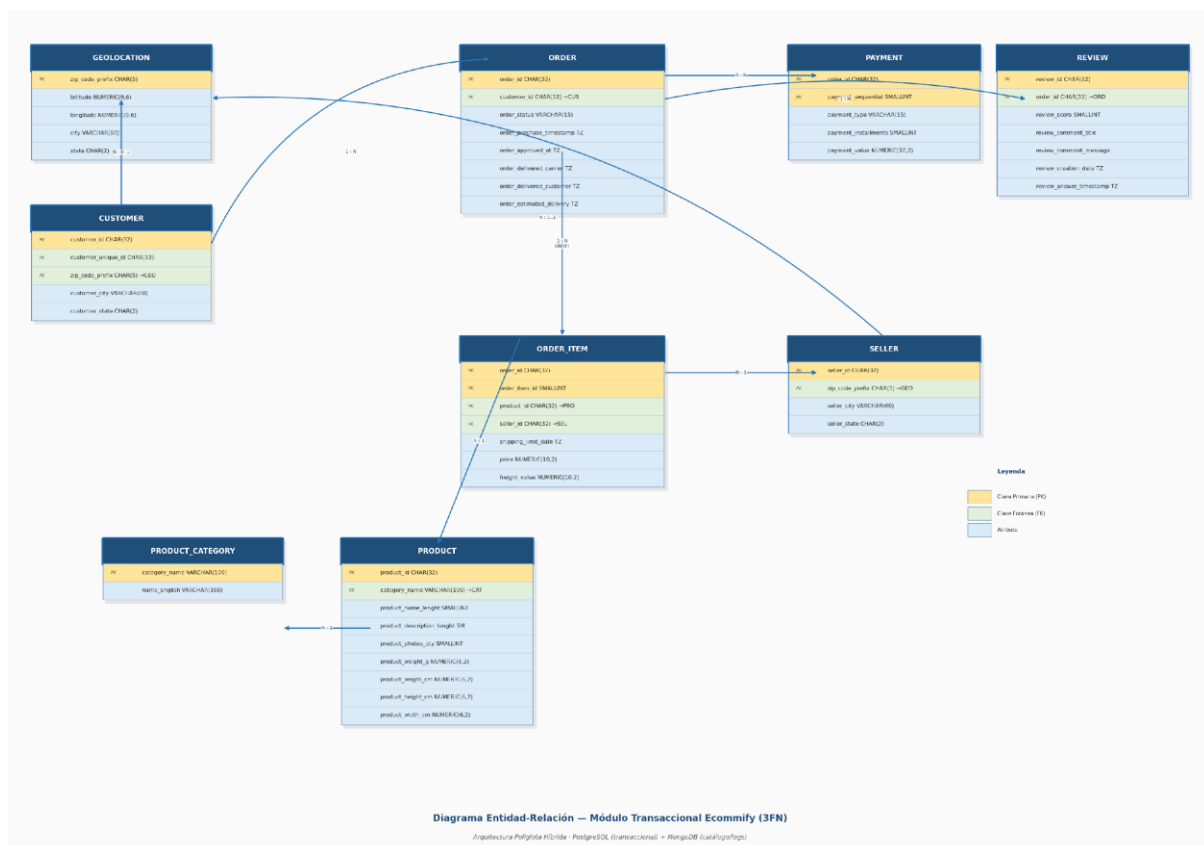


Ilustración 1. Módulo Transaccional Ecommify (3FN)

Descripción Detallada de Entidades y Atributos

CUSTOMER

Representa la cuenta de compra de un usuario en la plataforma. Una misma persona física puede tener múltiples registros de customer_id (uno por sesión de compra), pero se

identifica de forma única a través de `customer_unique_id`. Esta distinción fue crucial para resolver la violación de 1FN detectada en `OrdersFull`, donde la información del cliente se repetía en cada fila de ítem de la orden.

Tabla 3. Descripción de customer

Atributo	Restricción	Descripción	Forma Normal
customer_unique_id	PK CHAR(32)	Identidad real del comprador. Permite rastrear múltiples compras del mismo usuario.	1FN: sin este atributo no se distingue identidad real de sesión.
customer_id	NOT NULL CHAR(32)	Identificador de sesión de compra (hash MD5).	—
zip_code_prefix	FK -> GEOLOCATION CHAR(5)	Prefijo postal de 5 dígitos. FK hacía tabla de coordenadas.	3FN: city/state dependían del zip, no del customer_unique_id .
customer_city	VARCHAR(60)	Ciudad del cliente. Desnormalización de conveniencia sobre dato ya presente.	3FN (desnorm. controlada)
customer_state	CHAR(2)	Unidad Federativa de Brasil (ej. SP, MG, RJ).	3FN: mismo caso que <code>customer_city</code> .

ORDER

Representa un pedido con ciclo de vida completo. Es la entidad central del módulo transaccional y contiene los timestamps de cada hito del proceso logístico. La separación de `ORDER` como entidad independiente resuelve la violación de 1FN más evidente en `OrdersFull`: la información del pedido se repetía en todas las filas de ítems asociados a esa orden.

Tabla 4. Descripción de order

Atributo	Restricción	Descripción	Forma Normal
order_id	PK CHAR(32)	Identificador único del pedido (hash MD5).	—
customer_unique_id	FK -> CUSTOMER NOT NULL CHAR(32)	Asociar el pedido con la cuenta de compra.	1FN: en OrdersFull este valor se repetía por cada ítem de la orden.
order_status	NOT NULL VARCHAR(15) CHECK (8 valores)	Estado del ciclo de vida: created, approved, invoiced, processing, shipped, delivered, canceled, unavailable.	3FN: valor de catálogo repetido literalmente en cada fila de OrdersFull.
order_purchase_time stamp	TIMESTAMPTZ NOT NULL	Momento exacto de la compra con zona horaria UTC.	1FN: se repetía en cada ítem de la misma orden.
order_approved_at	TIMESTAMPTZ NULLABLE	Momento de aprobación del pago.	1FN
order_delivered_carrer_date	TIMESTAMPTZ NULLABLE	Entrega de la orden al transportista.	1FN
order_delivered_customer_date	TIMESTAMPTZ NULLABLE	Entrega efectiva al cliente final.	1FN
order_estimated_delivery_date	TIMESTAMPTZ NOT NULL	Fecha de entrega prometida al cliente.	1FN

ORDER_ITEM

Es la entidad débil central del esquema. Su PK compuesta (order_id, order_item_id) fue precisamente el ejemplo de violación de 2FN: price y freight_value dependen del ítem completo, mientras que los timestamps de la orden dependen solo de order_id. Al separar esta entidad se elimina la ambigüedad y se garantiza que cada atributo dependa de la clave completa.

Tabla 5. Descripción de order_item

Atributo	Restricción	Descripción	Forma Normal
----------	-------------	-------------	--------------

order_id	PK (parte 1), FK -> ORDER CHAR(32)	Referencia a la orden contenedora.	2FN: forma parte de la clave compuesta.
order_item_id	PK (parte 2) SMALLINT	Número de secuencia del ítem dentro de la orden.	2FN: forma parte de la clave compuesta.
product_id	FK -> PRODUCT NOT NULL CHAR(32)	Producto incluido en este ítem.	2FN: atributos del producto dependían solo de product_id, no de la PK compuesta.
seller_id	FK -> SELLER NOT NULL CHAR(32)	Vendedor que despacha este ítem específico.	3FN: datos del vendedor dependían transitivamente de seller_id.
shipping_limit_date	TIMESTAMP TZ NOT NULL	Fecha límite de envío para el vendedor.	2FN: depende de la PK compuesta completa.
price	NUMERIC(10,2) NOT NULL, CHECK > 0	Precio del ítem en BRL.	2FN: depende del ítem completo (order_id + order_item_id), no solo de product_id.
freight_value	NUMERIC(10,2) NOT NULL, CHECK >= 0	Flete del ítem en BRL. Puede ser cero.	2FN: idem price.

PAYMENT

Registra los métodos de pago de una orden. La separación de PAYMENT como entidad independiente resuelve una doble violación de 3FN: el tipo de pago era un literal repetido en OrdersFull, y una orden puede pagarse con múltiples métodos simultáneamente, lo que generaba múltiples filas con información de la orden duplicada.

Tabla 6. Descripción de payment

Atributo	Restricción	Descripción	Forma Normal
----------	-------------	-------------	--------------

order_id	PK (parte 1), FK -> ORDER CHAR(32)	Orden a la que corresponde el pago.	3FN: separado de ORDER porque una orden puede tener múltiples métodos de pago.
payment_sequential	PK (parte 2) SMALLINT	Número de secuencia del pago dentro de la orden.	—
payment_type	NOT NULL VARCHAR(15) CHECK (5 valores)	Método de pago: credit_card, debit_card, boleto, voucher, not_defined.	3FN: literal repetido en cada fila de OrdersFull.
payment_installments	SMALLINT NOT NULL, CHECK >= 1	Número de cuotas. Mínimo 1 (pago único).	—
payment_value	NUMERIC(10,2) NOT NULL, CHECK > 0	Monto del pago en BRL.	—

REVIEW

Captura la evaluación post-entrega del cliente. En OrdersFull, los datos de reseña estaban mezclados con los datos de ítems de la orden, generando una violación de 3FN: el review_score y los comentarios dependen de la relación cliente-orden, no de los ítems individuales comprados.

Tabla 7. Descripción de review

Atributo	Restricción	Descripción	Forma Normal
review_id	PK CHAR(32)	Identificador único de la resina (hash MD5).	—
order_id	FK -> ORDER NOT NULL CHAR(32)	Orden evaluada. CASCADE DELETE al eliminar la orden padre.	3FN: en OrdersFull los datos de reseña dependían de order_id mezclado con ítems.

review_score	SMALLINT NOT NULL, CHECK 1-5	Puntuación Likert del cliente. Escala cerrada de satisfacción.	3FN: atributo de evaluación post-entrega, no del ítem en sí.
review_comment_title	VARCHAR(50) NULLABLE	Título del comentario (max. 26 chars observadas en dataset).	—
review_comment_message	VARCHAR(300) NULLABLE	Cuerpo del comentario (max. 208 chars observados).	—
review_creation_date	TIMESTAMP Z NOT NULL	Fecha de solicitud de reseña por Olist.	—
review_answer_timestamp	TIMESTAMP Z NULLABLE	Fecha de respuesta del cliente a la solicitud.	—

PRODUCT

Representa los artículos del marketplace. Los nombres originales fueron anonimizados por Olist. Los atributos físicos (peso, dimensiones) presentaban una violación de 2FN en OrdersFull: dependían solo de `product_id` y no de la clave compuesta. En el modelo conceptual, PRODUCT se mantiene en PostgreSQL para las métricas transaccionales, mientras el catálogo enriquecido (descripción, fotos, especificaciones por categoría) se almacena en MongoDB.

Tabla 8. Descripción de product

Atributo	Restricción	Descripción	Forma Normal
product_id	PK CHAR(32)	Identificador único del producto (hash MD5). Nombre anonimizado por Olist.	—
category_name	FK -> PRODUCT_CATEGORY VARCHAR(100)	Categoría del producto en portugués. FK que resuelve dependencia transitiva de 3FN.	3FN: <code>name_english</code> dependía de <code>category_name</code> , no de <code>product_id</code> .

product_name_length	SMALLINT NULLABLE, CHECK >= 0	Longitud del nombre original del producto.	—
product_description_length	SMALLINT NULLABLE, CHECK >= 0	Longitud de la descripción del producto.	—
product_photos_qty	SMALLINT NULLABLE, CHECK >= 0	Cantidad de fotos del producto (max. 12 observado en dataset).	—
product_weight_g	NUMERIC(8,2) NULLABLE, CHECK > 0	Peso en gramos (max. 10,550 g observado).	2FN: dependía solo de product_id dentro de la PK compuesta.
product_length_cm	NUMERIC(6,2) NULLABLE, CHECK > 0	Longitud en centímetros.	2FN
product_height_cm	NUMERIC(6,2) NULLABLE, CHECK > 0	Alto en centímetros.	2FN
product_width_cm	NUMERIC(6,2) NULLABLE, CHECK > 0	Ancho en centímetros.	2FN

PRODUCT_CATEGORY

Esta tabla surge específicamente de la violación de 3FN detectada: product_category_name_english dependía de product_category_name, y no de product_id. Separar la tabla de categorías elimina esa dependencia transitiva. Con solo 71 categorías distintas y alta estabilidad, usa una clave natural (category_name) en lugar de una clave sustituta, lo que simplifica las FKs desde PRODUCT.

Tabla 9. Descripción de product_category

Atributo	Restricción	Descripción	Forma Normal
----------	-------------	-------------	--------------

category_name	PK VARCHAR(100) (clave natural)	Nombre de la categoría en portugués brasileño. Estable y con solo 71 valores distintos.	3FN: era una dependencia transitiva dentro de PRODUCT en OrdersFull.
name_english	NOT NULL VARCHAR(100)	Traducción oficial al inglés de la categoría.	3FN: dependía de category_name, no de product_id. Motivación directa de esta tabla.

SELLER

Representa a los vendedores registrados en la plataforma. El patrón de violación de 3FN es idéntico al de CUSTOMER: city y state dependen transitivamente de zip_code_prefix, no de seller_id. Ambas entidades comparten la solución: FK hacia la tabla GEOLOCATION.

Tabla 10. Descripción de seller

Atributo	Restricción	Descripción	Forma Normal
seller_id	PK CHAR(32)	Identificador único del vendedor (hash MD5).	—
zip_code_prefix	FK -> GEOLOCATION CHAR(5)	Código postal del vendedor. Resuelve la dependencia transitiva de 3FN.	3FN: city y state dependían del zip, no del seller_id.
seller_city	NOT NULL VARCHAR(60)	Ciudad del vendedor (desnormalización de conveniencia).	3FN: dependencia transitiva seller_id -> zip -> city.
seller_state	NOT NULL CHAR(2)	Estado (UF) del vendedor.	3FN: idem seller_city.

GEOLOCATION

Esta tabla resuelve la violación de 1FN del dataset original: el archivo olist_geolocation dataset.csv contiene múltiples filas por zip_code_prefix (hasta 141 registros para un mismo

código), lo que impedía definir una clave primaria válida. La solución fue consolidar las coordenadas mediante promedio ponderado de latitud/longitud y la ciudad/estado más frecuente por código postal, resultando en una fila única por ZIP prefix.

Tabla 11. Descripción de geolocation

Atributo	Restricción	Descripción	Forma Normal
zip_code_prefix	PK CHAR(5) (tras agregación)	Prefijo postal de 5 dígitos. Único por fila después de consolidar duplicados.	1FN: el dataset original viola 1FN con múltiples filas por ZIP.
latitude	NUMERIC(9,6)) NOT NULL	Latitud representativa del área postal (promedio ponderado). 6 decimales aprox. 0.1 m de precisión.	—
longitude	NUMERIC(9,6)) NOT NULL	Longitud representativa del área postal.	—
city	VARCHAR(60)) NOT NULL	Ciudad asociada al código postal (moda estadística del dataset).	—
state	CHAR(2) NOT NULL	Estado (UF) asociado al código postal.	—

Relaciones y Cardinalidades

Las relaciones del modelo conceptual se derivan directamente de cómo OrdersFull combinaba entidades en una sola tabla. Cada clave foránea (FK) en el esquema normalizado representa una relación que en OrdersFull era una columna repetida sin integridad referencial explícita:

Tabla 12. Relaciones y cardinalidades

Relación	Cardinalidad	Justificación (vínculo con análisis OrdersFull)
----------	--------------	---

CUSTOMER -> ORDER	1 : N (0..inf)	En OrdersFull, el customer_unique_id se repetía por cada fila de pedido. Violación de 1FN resuelta separando ORDER. Un cliente puede tener cero o más órdenes.
ORDER -> ORDER_ITEM	1 : N (1..inf), entidad débil	El ejemplo central de 2FN: PK compuesta (order_id, order_item_id). No existe orden sin al menos un ítem (cardinalidad mínima 1).
ORDER -> PAYMENT	1 : N (0..inf)	Una orden puede pagarse con múltiples métodos simultáneos. payment_type repetido en OrdersFull es violación de 3FN.
ORDER -> REVIEW	1 : N (0..inf)	Los datos de reseña estaban mezclados con ítems en OrdersFull (violación de 3FN por entidades mezcladas). Una orden puede no tener reseña.
ORDER_ITEM -> PRODUCT	N : 1	Los atributos físicos del producto dependían solo de product_id dentro de la PK compuesta: violación de 2FN. Muchos ítems referencian el mismo producto.
ORDER_ITEM -> SELLER	N : 1	Los datos del vendedor (city, state) dependían transitivamente de seller_id: violación de 3FN. Un vendedor puede despachar múltiples ítems.
PRODUCT -> PRODUCT_CATEG ORY	N : 1	name_english dependía de category_name, no de product_id: dependencia transitiva clásica de 3FN. Muchos productos pertenecen a la misma categoría.
CUSTOMER -> GEOLOCATION	N : 0..1	city y state del cliente dependían transitivamente de zip_code_prefix: violación de 3FN. La geolocalización puede no existir para todos los ZIP.
SELLER -> GEOLOCATION	N : 0..1	Mismo patrón de dependencia transitiva que CUSTOMER -> GEOLOCATION. Mismo ZIP prefix puede ser de cliente o vendedor.

Restricciones de Negocio Identificadas

Las siguientes restricciones son verificables a nivel de base de datos mediante PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK y NOT NULL, garantizando la integridad de los datos sin depender de la lógica de la aplicación:

Tabla 13. Restricciones del negocio

Tipo	Tabla · Atributo	Valor / Condición	Fundamento
PRIMAR Y KEY	Todas · clave primaria	—	Unicidad requerida por 1FN: cada fila debe ser identificable unívocamente. Los IDs son hashes MD5 de 32 chars: se usa CHAR(32) por longitud fija (más eficiente en índices que VARCHAR).
FOREIGN KEY RESTRICT	order · customer_unique_id -> customer	RESTRICT	Integridad referencial: no eliminar un cliente con órdenes activas (dato histórico transaccional).
FOREIGN KEY RESTRICT	order_item · order_id -> order	RESTRICT	La entidad débil no puede existir sin su entidad fuerte. Garantía del modelo ER.
FOREIGN KEY RESTRICT	order_item · product_id -> product	RESTRICT	Coherencia del catálogo: un ítem siempre referencia un producto real existente.
FOREIGN KEY CASCADE	review · order_id -> order	CASCADE DELETE	Una reseña sin orden pierde sentido semántico completo; se elimina en cascada si la orden es eliminada.
UNIQUE	customer · customer_unique_id	Valor único en tabla	Garantiza que la identidad real del comprador (customer_unique_id) no se duplique en el sistema.
CHECK dominio cerrado	order · order_status	8 valores: created, approved, invoiced, processing, shipped, delivered, canceled, unavailable	3FN: evita valores fuera del dominio controlado. El análisis de OrdersFull identificó exactamente estos 8 estados.
CHECK dominio cerrado	payment · payment_type	5 valores: credit_card, debit_card, boleto, voucher, not_defined	3FN: mismo criterio que order_status. Los 5 métodos cubren el 100% del dataset Olist.

CHECK rango Likert	review · review_score	BETWEEN 1 AND 5	Escala Likert estándar con dominio cerrado y conocido. Valores fuera de rango son errores de datos.
CHECK positivo estricto	order_item · price	> 0	Un precio negativo sería un error de dato, no un descuento. Los descuentos se modelan como ajustes separados.
CHECK no negativo	order_item · freight_value	>= 0	El flete puede ser cero (envío gratuito como incentivo comercial), pero nunca negativo.
CHECK cuotas	payment · payment_installments	>= 1	El pago mínimo es en una cuota. Cero cuotas no tienen sentido comercial.
NOT NULL crítico	order · order_purchase_timestamp, customer_unique_id	NOT NULL	Campos mínimos para que una orden sea válida y auditable en el sistema transaccional.
NUMERIC C precision	price, freight_value, payment_value	NUMERIC(10, 2)	Evita errores de representación de punto flotante (FLOAT/DOUBLE) en contextos financieros. Exactitud aritmética absoluta.

Tipos de Datos Avanzados en PostgreSQL

El segundo notebook de investigación profundizó en el uso de tipos avanzados de PostgreSQL aplicados al dataset de Ecommify. Estos tipos no solo optimizan el almacenamiento, sino que habilitan operadores especializados que simplifican consultas que de otro modo requerirían múltiples JOINS o procesamiento en la capa de aplicación:

Tabla 14. Tipo de datos avanzados

Tipo Avanzado	Campo en Esquema	Índice	Operadores Clave	Justificación Técnica y de Negocio
JSONB	product_specifications	GIN	->, ->>, @>	7 columnas planas (peso, dimensiones, fotos) -> 1 documento

				semiestructurado extensible por categoría. Sin ALTER TABLE al agregar nuevas specs. Permite búsquedas dentro del JSON con índice GIN.
JSONB	payment_info (en orders)	GIN	->, ->>, jsonb_array_elements()	Consolida múltiples métodos de pago de una orden en un único documento. Facilita consultas de análisis de mix de medios de pago sin JOIN adicional a tabla payment.
TEXT[]	product_photos	GIN	array_length(), @>, ANY()	URLs de fotos del CDN almacenadas como array. Son datos multivaluados que pertenecen semánticamente al producto y siempre se consultan juntas. Evita tabla product_photos separada con JOIN.
TSTZRANGE	promotion_period	GIST	@> (contención), && (solapamiento)	Modela el periodo activo de la orden [purchase_timestamp, estimated_delivery). Permite consultar órdenes activas en una fecha específica o durante una campaña con un solo operador indexado.
ENUM	order_status	B-Tree	=, IN, ORDER BY	Dominio cerrado de 8 valores. Validación automática a nivel DB (PostgreSQL rechaza valores inválidos). Almacenamiento interno como entero: más eficiente que TEXT y autodocumentado.
POINT	customer_location	GIST	<-> (distancia euclidiana)	Latitud y longitud son inseparables -- representan UN punto en el espacio. POINT permite el operador de distancia <-> directamente, habilitando análisis de proximidad cliente-vendedor.

Rol de MongoDB en la Arquitectura Políglota Híbrida

Si bien el presente entregable se enfoca en el módulo transaccional de PostgreSQL, la arquitectura políglota implica que ciertos datos se gestionan en MongoDB. Las decisiones de asignación son las siguientes:

- Catálogo de productos enriquecido: los documentos de producto en MongoDB contienen el `product_id` como referencia al registro PostgreSQL, más atributos extendidos variables por categoría (descripciones largas, especificaciones técnicas, URLs de medios, tags, variantes). Esta flexibilidad de esquema es imposible en el modelo relacional sin un diseño Entity-Attribute-Value (EAV) altamente complejo.
- Logs de eventos operacionales: eventos como "producto visto", "carrito abandonado", "clic en categoría" generan alto volumen de escritura con esquema variable. MongoDB admite colecciones con documentos de distinta forma, ideal para este caso de uso.
- Sesiones de usuario y carritos temporales: datos con TTL (Time-To-Live) natural que se crean y destruyen frecuentemente, donde el modelo orientado a documentos de MongoDB es más eficiente que tablas relacionales con alta tasa de INSERT/DELETE.

La sincronización entre ambos motores se realiza a través del `product_id` como clave compartida: PostgreSQL mantiene la referencia en `order_item.product_id`, y MongoDB mantiene el documento enriquecido en su colección `products`. Esta separación garantiza que las transacciones ACID del módulo operacional no se vean afectadas por la flexibilidad del catálogo, y permite que cada motor escale de forma independiente según las necesidades del negocio.

Diseño Lógico — Módulo PostgreSQL

Esta sección presenta el diseño lógico completo del módulo transaccional de Ecommify implementado en PostgreSQL sobre Supabase. El esquema relacional está normalizado hasta la Tercera Forma Normal (3FN), aplica tipos de datos avanzados justificados técnicamente e incorpora estrategias de particionamiento e indexación para soportar cargas híbridas OLTP/OLAP.

Esquema Relacional Normalizado (3FN)

El esquema relacional deriva directamente del diseño conceptual presentado en la sección anterior. Las nueve entidades identificadas se implementan como tablas en el esquema público de PostgreSQL, con claves primarias, foráneas y constraints definidos a nivel de base de datos. A continuación, se presenta la especificación tabla por tabla.

Especificación Tabla por Tabla

Tabla: geolocation

Coordenadas geospaciales consolidadas por prefijo postal (1 fila por ZIP tras agregar duplicados del dataset original).

Tabla 15. Especificación geolocation

Columna	Tipo de Dato	Clave	Constraint	Descripción
zip_code_prefix	CHAR(5)	PK	NOT NULL	Prefijo postal único (5 dígitos). Clave natural.
latitude	NUMERIC(9,6)		NOT NULL	Latitud promedio del área postal.
longitude	NUMERIC(9,6)		NOT NULL	Longitud promedio del área postal.
city	VARCHAR(60)		NOT NULL	Ciudad asociada al ZIP (moda estadística).

state	CHAR(2)	NOT NULL	Estado (UF) de Brasil.
-------	---------	----------	------------------------

Tabla: product_category

Catálogo de categorías en portugués con traducción al inglés. 71 valores estables; usa clave natural.

Tabla 16. Especificación product_category

Columna	Tipo de Dato	Clave	Constraint	Descripción
category_name	VARCHAR(100)	PK	NOT NULL	Nombre en portugués. Clave natural estable (71 valores).
name_english	VARCHAR(100)		NOT NULL	Traducción oficial al inglés.

Tabla: customer

Cuenta de compra. Una persona puede tener múltiples customer_id (uno por sesión), identificada unívocamente por customer_unique_id.

Tabla 17. Especificación customer

Columna	Tipo de Dato	Clave	Constraint	Descripción
customer_unique_id	CHAR(32)	PK	UNIQUE NOT NULL	Identidad real del comprador (cross-session).
customer_id	CHAR(32)		NOT NULL	Hash MD5 de sesión de compra.
zip_code_prefix	CHAR(5)	FK→geo location	ON DELETE SET NULL	Prefijo postal. Resuelve dependencia transitiva 3FN.
customer_city	VARCHAR(60)			Desnormalización controlada de conveniencia.
customer_state	CHAR(2)			UF del cliente.

Tabla: seller

Vendedores registrados en la plataforma con geolocalización por código postal.

Tabla 18. Especificación seller

Columna	Tipo de Dato	Clave	Constraint	Descripción
seller_id	CHAR(32)	PK	NOT NULL	Hash MD5 del vendedor.
zip_code_prefix	CHAR(5)	FK→geolocation	ON DELETE SET NULL	Prefijo postal del vendedor.
seller_city	VARCHAR(60)		NOT NULL	Ciudad del vendedor.
seller_state	CHAR(2)		NOT NULL	Estado (UF) del vendedor.

Tabla: product

Artículos del marketplace con atributos físicos y campos JSONB/TEXT[] para especificaciones y fotos.

Tabla 19. Especificación product

Columna	Tipo de Dato	Clave	Constraint	Descripción
product_id	CHAR(32)	PK	NOT NULL	Hash MD5. Nombre anonimizado por Olist.
category_name	VARCHAR(100)	FK→product_category	ON DELETE SET NULL	Categoría en portugués. FK resuelve dependencia 3FN.
product_name_length	SMALLINT		CHECK >= 0	Longitud del nombre original.
product_description_length	SMALLINT		CHECK >= 0	Longitud de la descripción.
product_photos_qty	SMALLINT		CHECK >= 0	Cantidad de fotos (max 12 observado).
product_weight_g	NUMERIC(8,2)		CHECK > 0	Peso en gramos.
product_length_cm	NUMERIC(6,2)		CHECK > 0	Longitud en cm.

product_height_cm	NUMERIC(6,2)	CHECK > 0	Alto en cm.
product_width_cm	NUMERIC(6,2)	CHECK > 0	Ancho en cm.
specifications	JSONB		Tipo avanzado. Ver sección D.3.
photo_urls	TEXT[]		Tipo avanzado. Ver sección D.3.

Tabla: orders

Pedido con ciclo de vida completo. Tabla particionada por fecha (ver sección D.4).

Incluye campo TSTZRANGE para análisis de períodos.

Tabla 20. Especificación orders

Columna	Tipo de Dato	Clave	Constraint	Descripción
order_id	CHAR(32)	PK	NOT NULL	Hash MD5 del pedido.
customer_unique_id	CHAR(32)	FK→customer	NOT NULL, ON DELETE RESTRICT	Cuenta de compra del cliente.
order_status	order_status_enum		NOT NULL	ENUM: 8 estados del ciclo de vida.
order_purchase_timestamp	TIMESTAMP TZ		NOT NULL	Momento de la compra (UTC).
order_approved_at	TIMESTAMP TZ		NULLABLE	Aprobación del pago.
order_delivered_carrier_date	TIMESTAMP TZ		NULLABLE	Entrega al transportista.
order_delivered_customer_date	TIMESTAMP TZ		NULLABLE	Entrega al cliente.
order_estimated_delivery_date	TIMESTAMP TZ		NOT NULL	Fecha estimada de entrega.
promotion_period	TSTZRANGE			Tipo avanzado. Ver sección D.3.
updated_at	TIMESTAMP TZ		DEFAULT now()	Mantenido por trigger. Ver sección D.5.

Tabla: order_item

Entidad débil de ORDER. PK compuesta (order_id, order_item_id). Vincula orden, producto y vendedor con precio y flete por ítem.

Tabla 21. Especificación order_item

Columna	Tipo de Dato	Clave	Constraint	Descripción
order_id	CHAR(32)	PK(1), FK→order s	NOT NULL, ON DELETE RESTRICT	Orden contenedora.
order_item_id	SMALLINT	PK(2)	NOT NULL	Secuencia del ítem dentro de la orden.
product_id	CHAR(32)	FK→prod uct	NOT NULL, ON DELETE RESTRICT	Producto del ítem.
seller_id	CHAR(32)	FK→seller	NOT NULL, ON DELETE RESTRICT	Vendedor que despacha.
shipping_limit_date	TIMESTAMP TZ		NOT NULL	Fecha límite de envío.
price	NUMERIC(10, 2)		NOT NULL, CHECK > 0	Precio del ítem en BRL.
freight_value	NUMERIC(10, 2)		NOT NULL, CHECK >= 0	Flete del ítem en BRL.

Tabla: payment

Métodos de pago de una orden. PK compuesta permite múltiples métodos simultáneos por orden.

Tabla 22. Especificación payment

Columna	Tipo de Dato	Clave	Constraint	Descripción
order_id	CHAR(32)	PK(1), FK→or ders	NOT NULL, ON DELETE RESTRICT	Orden asociada al pago.
payment_seque ntial	SMALLINT	PK(2)	NOT NULL	Secuencia del pago dentro de la orden.

payment_type	VARCHAR(15)	NOT NULL, CHECK (5 valores)	Método: credit_card, debit_card, boleto, voucher, not_defined.
payment_installments	SMALLINT	NOT NULL, CHECK >= 1	Número de cuotas (mínimo 1).
payment_value	NUMERIC(10,2)	NOT NULL, CHECK > 0	Monto del pago en BRL.

Tabla: review

Evaluación post-entrega del cliente. ON DELETE CASCADE: se elimina si se elimina la orden padre.

Tabla 23. Especificación review

Columna	Tipo de Dato	Clave	Constraint	Descripción
review_id	CHAR(32)	PK	NOT NULL	Hash MD5 de la reseña.
order_id	CHAR(32)	FK→orders	NOT NULL, ON DELETE CASCADE	Orden evaluada. CASCADE: reseña sin orden pierde sentido.
review_score	SMALLINT		NOT NULL, CHECK 1-5	Puntuación Likert de satisfacción.
review_comment_title	VARCHAR(50)		NULLABLE	Título del comentario.
review_comment_message	VARCHAR(300)		NULLABLE	Cuerpo del comentario.
review_creation_date	TIMESTAMP TZ		NOT NULL	Solicitud de reseña enviada por Olist.
review_answer_timestamp	TIMESTAMP TZ		NULLABLE	Fecha de respuesta del cliente.

Aplicación de Tipos de Datos Avanzados

La siguiente tabla justifica la selección de cada tipo avanzado de PostgreSQL, indicando el campo donde se aplica, el índice recomendado, los operadores habilitados y el beneficio técnico y de negocio concreto para Ecommify.

Tabla 24. Aplicación datos avanzados

Tipo	Campo	Índice	Operadores clave	Justificación técnica
JSONB	product.specifications	GIN	->, ->>, @>	Consolida 7 columnas planas de specs físicas en un documento extensible por categoría. Sin ALTER TABLE al agregar nuevos atributos. GIN acelera búsquedas dentro del JSON.
TEXT[]	product.photo_urls	GIN	array_length(), @>, ANY()	URLs de fotos del CDN son datos multivaluados que siempre se consultan junto al producto. Evita tabla photo separada con JOIN adicional.
TSTZRANGE	orders.promotion_period	GIST	@> (contención), && (solapamiento)	Modela el período activo de una orden [purchase_timestamp, estimated_delivery). Permite consultar órdenes activas en una fecha con un operador indexado, sin rango manual entre dos columnas.
ENUM (order_status_enum)	orders.order_status	B-Tree	=, IN, ORDER BY	Dominio cerrado de 8 valores. PostgreSQL rechaza valores inválidos automáticamente. Almacenamiento interno como entero: más eficiente que TEXT en índices y ORDER BY.
POINT	customer/seller (view)	GIST	<-> (distancia euclidiana)	Latitud y longitud son inseparables — representan un punto en el espacio. POINT con índice GIST habilita consultas de proximidad vendedor-cliente en análisis de costos de envío, sin extensión PostGIS.

Extensiones de PostgreSQL Utilizadas

Se evaluaron las siguientes extensiones en función de los requisitos operacionales de Ecommify. La decisión final prioriza extensiones disponibles en Supabase Free Tier sin configuración adicional.

Tabla 25. Extensiones utilizadas

Extensión	Decisión	Caso de uso en Ecommify	Justificación
-----------	----------	-------------------------	---------------

pg_trgm	ADO PTA R	Búsqueda tolerante a errores tipográficos en nombres de productos y categorías.	Habilitada por defecto en Supabase. Crea índices GIN/GIST sobre trigramas. Permite similitud fuzzy (similarity() > 0.3) sin dependencia externa. Alta relevancia para el buscador de productos.
pgcrypto	ADO PTA R	Generación y validación de hashes MD5 para IDs (customer_unique_id, order_id, product_id, etc.).	Disponible en Supabase. Permite gen_random_uuid() y digest() nativamente en la base de datos, sin depender de la capa de aplicación para generar IDs.
PostGIS	DIFE RIR	Optimización de costos de envío basados en distancia geoespacial vendedor-cliente.	No disponible en Supabase Free Tier. El tipo POINT nativo de PostgreSQL con operador <-> cubre las consultas de proximidad básicas. PostGIS se incorporará en fase de escalamiento al plan Pro.
hstore	DES CAR TAR	Alternativa a JSONB para almacenamiento de pares clave-valor.	JSONB supera a hstore en todas las dimensiones relevantes: operadores más ricos, soporte de tipos anidados, índices GIN más eficientes. hstore queda obsoleto para nuevos diseños en PostgreSQL 9.4+.

Estrategia de Particionamiento

La tabla orders es la candidata principal para particionamiento dado su volumen proyectado y su patrón de acceso temporal predominante (consultas filtradas por rango de fecha). Se implementa particionamiento por rango (RANGE PARTITIONING) sobre order_purchase_timestamp, siguiendo el patrón hot/cold partitions. Esquema:

```

CREATE TABLE orders (
    order_id                CHAR(32)                NOT NULL,
    customer_id             CHAR(32)                NOT NULL,
    order_status            order_status_enum       NOT NULL,
    order_purchase_timestamp TIMESTAMPTZ            NOT NULL,
    order_approved_at       TIMESTAMPTZ,
    order_delivered_carrier_date TIMESTAMPTZ,
    order_delivered_customer_date TIMESTAMPTZ,
    order_estimated_delivery_date TIMESTAMPTZ       NOT NULL,
    promotion_period        TSTZRANGE,
    updated_at              TIMESTAMPTZ            DEFAULT now(),
    PRIMARY KEY (order_id, order_purchase_timestamp)
) PARTITION BY RANGE (order_purchase_timestamp);

-- Partición hot: año actual (acceso frecuente)
CREATE TABLE orders_2024 PARTITION OF orders
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');

-- Partición cold: datos históricos comprimidos
CREATE TABLE orders_2016_2023 PARTITION OF orders
    FOR VALUES FROM ('2016-01-01') TO ('2024-01-01');

```

Ilustración 2. Esquema DDL de particionamiento

Las tablas `order_item` y `payment` heredan el beneficio del `partition pruning` a través de sus FKs hacia `orders`, ya que las consultas analíticas típicamente filtran por rango temporal de la orden. Se programa la creación de la partición del año siguiente cada diciembre mediante un job programado en `pg_cron`.

Triggers y Vistas Materializadas

Se definen los siguientes objetos de base de datos para mantener la consistencia y soportar cargas analíticas:

- Trigger `updated_at`: función `set_updated_at()` aplicada a `orders` mediante `BEFORE UPDATE FOR EACH ROW`. Garantiza que `updated_at` siempre refleje la última modificación sin depender de la capa de aplicación.
- Vista materializada `mv_sales_by_category_monthly`: agrega ingresos totales, cantidad de órdenes y valor promedio por categoría de producto y mes. Refresh semanal programado. Soporta dashboards analíticos sin impactar las tablas transaccionales.
- Vista materializada `mv_customer_segments`: segmenta clientes por frecuencia de compra (RFM simplificado: Recency, Frequency, Monetary). Refresh mensual. Insumo para campañas de marketing y análisis de retención.

Estrategia de Indexación

Más allá de los índices implícitos en las PKs, se definen los siguientes índices adicionales sobre las columnas de mayor cardinalidad y frecuencia de filtro:

Tabla 26. Estrategia de indexación

Índice	Tabla · Columna	Tipo	Justificación
<code>idx_orders_customer</code>	<code>orders.customer_unique_id</code>	B-Tree	Join frecuente <code>customer→orders</code> para historial de compras y análisis de retención.
<code>idx_orders_status</code>	<code>orders.order_status</code>	B-Tree	Filtros operacionales por estado (e.g., <code>shipped</code> , <code>pending</code>). Alta selectividad en consultas de gestión logística.
<code>idx_order_items_product</code>	<code>order_item.product_id</code>	B-Tree	Join crítico para análisis de ventas por producto. Alta frecuencia en consultas OLAP.
<code>idx_order_items_seller</code>	<code>order_item.seller_id</code>	B-Tree	Consultas de rendimiento por vendedor. Soporte para dashboard de sellers.
<code>idx_geolocation_zip</code>	<code>geolocation.zip_code_prefix</code>	B-Tree	Join geográfico <code>customer/seller→geolocation</code> con 1M+ filas. Sin índice, full scan en cada consulta de proximidad.

idx_product_specs_gin	product.specifications	GIN	Búsquedas dentro del documento JSONB (@>). Esencial para filtros de especificaciones técnicas.
idx_product_photos_gin	product.photo_urls	GIN	Operaciones de contención sobre arrays (@>, ANY). Útil para validar integridad de URLs.
idx_orders_period_gist	orders.promotion_period	GIST	Operadores de rango TSTZRANGE (@>, &&). Sin GIST, los operadores de rango no pueden usar índices.
idx_product_trgm	product.category_name	GIN (pg_trgm)	Búsqueda fuzzy tolerante a errores tipográficos en nombres de categoría mediante similarity().

Diseño Lógico Preliminar — Módulo MongoDB

Esta sección presenta el diseño preliminar del módulo NoSQL de Ecommify en MongoDB. La arquitectura políglota asigna a MongoDB los datos semi-estructurados y de alta variabilidad de esquema, complementando el módulo transaccional de PostgreSQL. El criterio de asignación se fundamenta en el principio de polyglot persistence: cada motor gestiona los datos para los que está optimizado.

Criterio de Asignación: PostgreSQL vs MongoDB

Tabla 27. Matriz de criterio

Entidad / Colección	Motor	Volumen	Patrón de acceso	Justificación
customer, order, order_item, payment, review	PostgreSQL	~500K registros	OLTP intensivo	Garantías ACID, integridad referencial y consistencia crítica. Datos altamente estructurados con esquema estable.
seller, geolocation, product_category	PostgreSQL	~1M registros	Referencia estática	Tablas de catálogo con alta estabilidad de esquema. Se benefician de índices B-Tree y JOINS relacionales.
products (catálogo enriquecido)	MongoDB	~33K docs, +20% anual	Lectura intensiva, esquema variable	Especificaciones técnicas difieren radicalmente por categoría. MongoDB permite atributos variables sin ALTER TABLE. Un electrónico requiere voltaje/garantía; una prenda, talla/material.
event_logs (clicks, carritos, vistas)	MongoDB	Alto volumen, +50% anual	Escritura masiva, TTL	Eventos con esquema variable y TTL natural. MongoDB admite colecciones heterogéneas ideal para este caso. No requiere integridad referencial estricta.
user_sessions (carritos temporales)	MongoDB	Variable, TTL 24h	Escritura/lectura frecuente	Datos efímeros con TTL. El modelo orientado a documentos con expiración automática (TTL Index) es más eficiente que tablas relacionales con alta tasa de INSERT/DELETE.

Colecciones Principales

Se identifican tres colecciones principales para el módulo MongoDB de Ecommify:

- **products:** Catálogo enriquecido de productos con especificaciones variables por categoría. Es la colección central del módulo NoSQL.
- **event_logs:** Registro de eventos operacionales (vistas de producto, clics, búsquedas, carritos abandonados). Alto volumen de escritura con TTL configurable.
- **user_sessions:** Sesiones activas y carritos de compra temporales. Documentos con TTL de 24 horas para expiración automática.

Esquema de Documentos (Ejemplos JSON)

Colección: products

El documento de producto en MongoDB referencia el `product_id` de PostgreSQL como clave de sincronización. Los atributos de `specifications` varían libremente por categoría sin requerir migraciones de esquema.


```

{
  "_id": "ObjectId(' ... ')",
  "product_id": "ffe9c82b9f56afcfd0dfe57d81de5f5a5",
  "category_name": "informatica_acessorios",
  "category_name_english": "computers_accessories",
  "name": "[Anonimizado por Olist]",
  "description": "Descripción completa del producto ... ",
  "photo_urls": [
    "https://cdn.ecommify.com/products/img1.jpg",
    "https://cdn.ecommify.com/products/img2.jpg"
  ],
  "specifications": {
    "weight_g": 300,
    "dimensions_cm": { "length": 20, "height": 5, "width": 15 },
    "connectivity": ["USB-A", "Bluetooth 5.0"],
    "compatibility": ["Windows", "macOS", "Linux"],
    "warranty_months": 12
  },
  "tags": ["periferico", "mouse", "inalambrico"],
  "avg_review_score": 4.3,
  "total_reviews": 128,
  "created_at": "ISODate('2024-01-15T10:00:00Z')",
  "updated_at": "ISODate('2024-06-20T14:30:00Z')"
}

```

Ilustración 3. Esquema products

Colección: event_logs

Registra eventos operacionales de usuarios. El esquema es intencionalmente flexible para acomodar distintos tipos de evento sin modificar la estructura de la colección.

```
{
  "_id": "ObjectId(' ... ')",
  "event_type": "product_view",
  "customer_unique_id": "d6d5a20df8c820cbce39fd49b34bd9ac",
  "session_id": "sess_abc123",
  "product_id": "ffe9c82b9f56afc0dfe57d81de5f5a5",
  "category": "informatica_acessorios",
  "metadata": {
    "source": "search_results",
    "position": 3,
    "search_query": "mouse inalambrico"
  },
  "timestamp": "ISODate('2024-06-20T14:32:10Z')",
  "ttl_expire": "ISODate('2024-09-20T14:32:10Z')"
}
```

Ilustración 4. Esquema event_logs

Colección: user_sessions

Almacena carritos de compra y sesiones activas con expiración automática mediante TTL Index de MongoDB. Evita el overhead de tablas relacionales con alta tasa de INSERT/DELETE para datos efímeros.

```

{
  "_id": "sess_abc123",
  "customer_unique_id": "d6d5a20df8c820cbce39fd49b34bd9ac",
  "cart": [
    {
      "product_id": "ffe9c82b9f56afc0dfe57d81de5f5a5",
      "quantity": 1,
      "unit_price": 89.90,
      "seller_id": "cca3071a3a8b5b6a ..."
    }
  ],
  "created_at": "ISODate('2024-06-20T14:30:00Z')",
  "expires_at": "ISODate('2024-06-21T14:30:00Z')"
}

```

Ilustración 5. Esquema user_sessions

Patrones de Modelado Aplicados

Tabla 28. Patrones aplicados

Patrón	Colección	Descripción y aplicación en Ecommify
Extended Reference	products	El documento de producto incluye product_id y category_name como referencias al esquema PostgreSQL (clave de sincronización). Las especificaciones y medios se almacenan embebidos en MongoDB, evitando JOINS costosos en consultas de catálogo.
Subset Pattern	products	avg_review_score y total_reviews se almacenan como campos calculados en el documento de producto (subconjunto de datos de reviews). Permite mostrar la valoración en listados sin consultar PostgreSQL ni calcular en tiempo real.
Bucket Pattern	event_logs	Los eventos se agrupan en documentos por sesión o por ventana temporal (e.g., un documento por hora de eventos de un usuario). Reduce el número de documentos y mejora el rendimiento de consultas de analítica de comportamiento.
TTL Pattern	user_sessions, event_logs	Índice TTL sobre expires_at/ttl_expire respectivamente. MongoDB elimina documentos expirados automáticamente sin intervención de la

		aplicación. user_sessions expiran en 24h; event_logs en 90 días.
Schema Versioning	products	Campo schema_version en cada documento de producto. Permite evolucionar el esquema de specifications de forma gradual sin migraciones globales. Los documentos con versiones antiguas coexisten con los nuevos mientras se migra incrementalmente.

Flujo de Sincronización PostgreSQL ↔ MongoDB

La clave de sincronización entre ambos motores es product_id (CHAR(32), hash MD5). Este campo existe tanto en PostgreSQL (product.product_id, order_item.product_id) como en MongoDB (products.product_id). El flujo de sincronización opera de la siguiente manera:

- Creación de producto: el registro base (product_id, category_name, dimensiones físicas) se inserta primero en PostgreSQL para garantizar integridad referencial. Inmediatamente después, el documento enriquecido se crea en MongoDB con el mismo product_id como clave compartida.
- Actualización de catálogo: cambios en atributos de specifications o photo_urls se aplican directamente en MongoDB (updateOne por product_id). No requieren coordinación con PostgreSQL salvo cambio de category_name (FK en PostgreSQL).
- Métricas de reseñas: un job periódico (cada 6 horas) agrega review_score desde PostgreSQL y actualiza avg_review_score y total_reviews en MongoDB. Se acepta eventual consistencia en estos campos calculados.
- Eliminación de producto: se elimina primero en PostgreSQL (verificando integridad referencial con order_item) y después en MongoDB. El orden garantiza que no existan referencias huérfanas en el módulo transaccional.

Decisiones arquitectónicas justificadas

Matriz de decisión por entidad

Tabla 29. Matriz de decisión

Módulo	CAP Prioridad	Naturaleza a Datos	Patrón Acceso	Motor	Justificación
CUSTOMER	CP	Estructurados	Transaccional	PostgreSQL	Datos personales sensibles. Integridad ACID. FK crítica con ORDER.
ORDER	CP	Estructurados	OLTP intensivo	PostgreSQL	Núcleo transaccional. ACID indispensable: evita pedidos duplicados. Particionado por fecha.
ORDER_ITEM	CP	Estructurados	OLTP / OLAP	PostgreSQL	Entidad débil con PK compuesta. FK hacia ORDER, PRODUCT, SELLER.
PAYMENT	CP	Estructurados	OLTP crítico	PostgreSQL	Operaciones financieras. Atomicidad estricta. Cero tolerancias a inconsistencias.
REVIEW	CP	Estructurados	OLAP / escritura periódica	PostgreSQL	Atributos estables (score, fechas). CASCADE con ORDER.
PRODUCT (core)	CP	Estructurados	Transaccional	PostgreSQL	Métricas transaccionales (peso, dimensiones). FK a PRODUCT_CATEGORY.
PRODUCT_CATEGORY	CP	Estructurados	Catálogo estático	PostgreSQL	Clave natural estable. Resuelve dependencia transitiva 3FN.
SELLER	CP	Estructurados	OLTP moderado	PostgreSQL	Datos fiscales y contractuales. FK con ORDER_ITEM.
GEOLOCATION	AP	Semi-estructurados	Lectura intensiva	PostgreSQL	1M registros, acceso por zip. Índice B-tree

		os			suficiente. Sin particionamiento.
PRODUCT_CATALOG	AP	Semi-estructurados	Lectura intensiva	MongoDB	Atributos variables por categoría. Esquema flexible. Sharding por categoría.
LOGS	AP	Semi-estructurados	Escritura masiva	MongoDB	TTL natural. Esquema variable. Alta disponibilidad.
USER_SESSIONS	AP	Semi-estructurados	Escritura/Lectura	MongoDB	Documentos efímeros con TTL 24h.

Análisis del Teorema CAP aplicado

Se evaluó cada módulo frente al Teorema CAP (Consistency, Availability, Partition Tolerance). Dado que la tolerancia a particiones (P) es inevitable en sistemas distribuidos, la decisión real es CP vs AP:

Prioridad CP (Consistencia > Disponibilidad)

Tabla 30. Prioridad CP

Módulo	Motivo
CUSTOMER	Datos sensibles requieren consistencia fuerte. Inconsistencia → datos duplicados o pérdida de identidad.
ORDER	Pedido duplicado o estado inconsistente → pérdida financiera y legal.
ORDER_ITEM	Integridad referencial con ORDER, PRODUCT, SELLER. Un ítem huérfano es ilegítimo.
PAYMENT	Operaciones financieras: tolerancia cero a inconsistencias. Double-spending inaceptable.
REVIEW	Una reseña debe corresponder exactamente a una orden.
SELLER	Datos contractuales requieren consistencia.

Prioridad AP (Disponibilidad > Consistencia)

Tabla 31. Prioridad AP

Módulo	Motivo
Catálogo de Productos (MongoDB)	Alta disponibilidad para consultas. Consistencia eventual tolerable: un producto puede aparecer segundos después de crearse.
Geolocalización	Consultas de proximidad y envío. Disponibilidad > consistencia estricta.
Logs de eventos	Escritura masiva. Si se pierde un evento de log, el sistema no falla.

Diagrama CAP

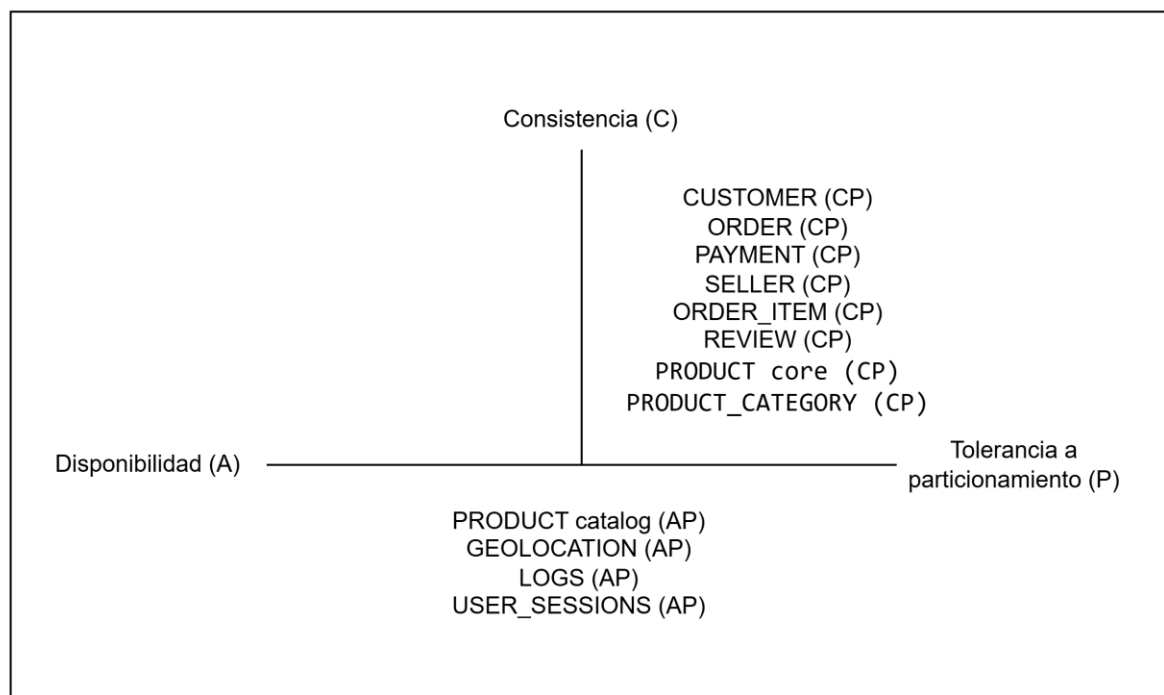


Ilustración 6. Distribución CAP

Trade-offs y estrategias de mitigación

Tabla 32. Trade-offs

Trade-off	Decisión	Riesgo	Mitigación
Normalización 3FN	Normalizar todo el	JOINS múltiples	Índices compuestos

Trade-off	Decisión	Riesgo	Mitigación
vs rendimiento consultas	módulo transaccional a 3FN	impactan rendimiento en consultas analíticas	(B-tree, GIN). Vistas materializadas para reportes frecuentes. Particionamiento por fecha en ORDER.
Persistencia políglota vs complejidad operativa	PostgreSQL + MongoDB	Dos motores que administrar, sincronización entre ellos	CDC con Debezium + Kafka para replicación asíncrona. Outbox Pattern para eventos críticos. product_id como clave compartida.
Tipos avanzados PostgreSQL	JSONB, TEXT[], TSTZRANGE, ENUM, POINT	Curva de aprendizaje del equipo; acoplamiento a características específicas de PostgreSQL	Todos los tipos son parte del estándar PostgreSQL ≥ 12 . Documentación extensa. Beneficio: reducción de JOINS y consultas más simples.
Desnormalización controlada	Mantener city/state en CUSTOMER y SELLER pese a FK a GEOLOCATION	Datos redundantes, posible inconsistencia si cambia GEOLOCATION	La redundancia es de conveniencia (dato ya presente al insertar). ON UPDATE CASCADE en FK para propagar cambios de ZIP.
Particionamiento en ORDER	RANGE por order_purchase_times tamp	Complejidad en migraciones y mantenimiento de particiones	Partition pruning automático de PostgreSQL. Tabla order_future para datos fuera de rango. Evaluar reparticionamiento a los 5M de filas.
Múltiples pagos por orden	PK compuesta (order_id, payment_sequential)	Consultas de agregación requieren SUM + GROUP BY	Indexar order_id en PAYMENT. Vista materializada v_order_total_paid.

Flujos de sincronización

Arquitectura Híbrida Propuesta

La arquitectura resultante combina ambos paradigmas en un modelo de persistencia políglota organizado en capas, donde cada motor cumple el rol para el cual está técnicamente optimizado. La interacción entre componentes se orquesta mediante un bus de eventos central y un mecanismo de Change Data Capture (CDC), garantizando coherencia entre las distintas fuentes de datos sin comprometer las ventajas individuales de cada tecnología.

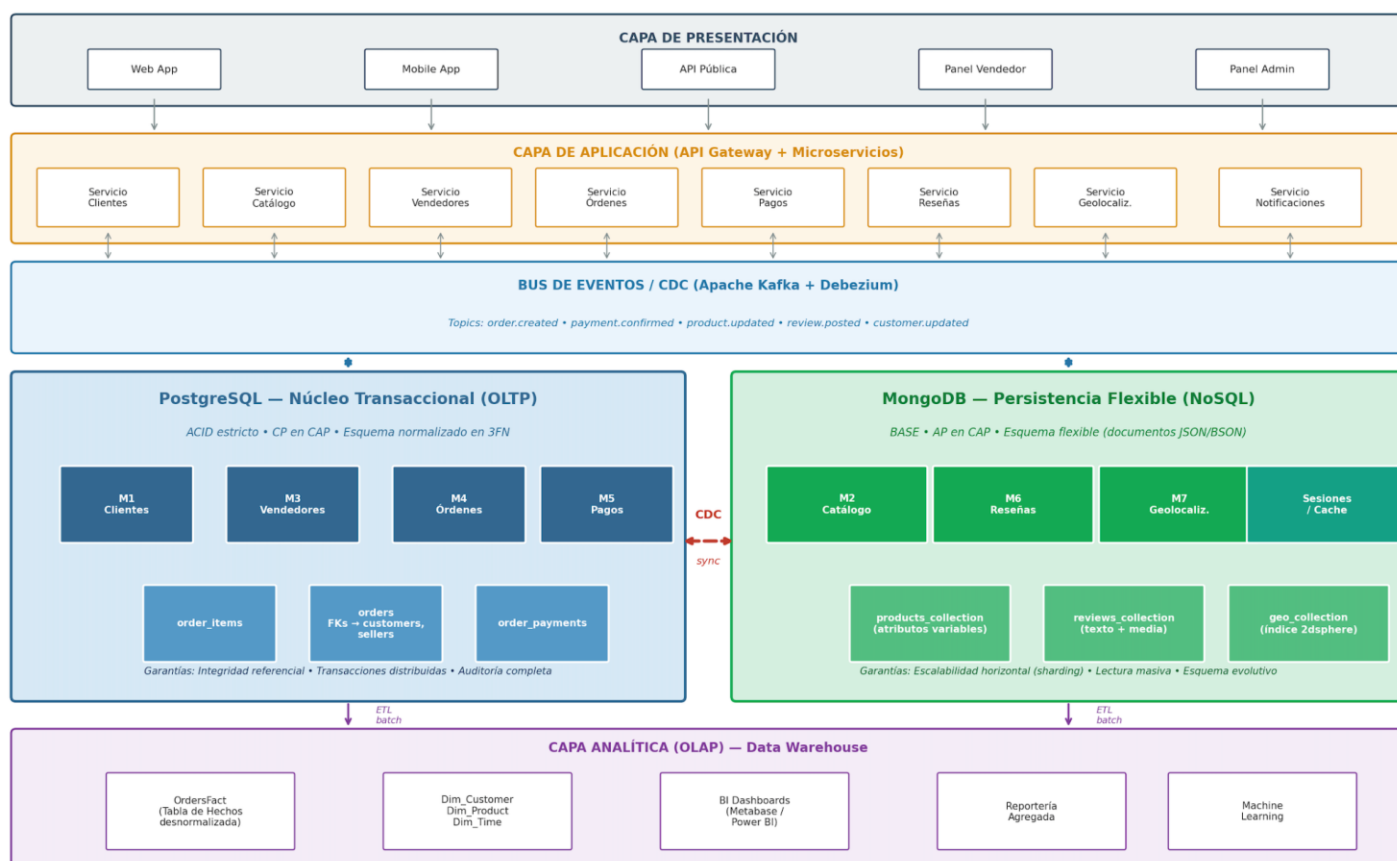


Ilustración 7. Arquitectura

Flujo A — Change Data Capture (CDC)

Debezium captura cambios del WAL de PostgreSQL y los publica en Kafka:

1. **INSERT/UPDATE/DELETE** en PostgreSQL → Write-Ahead Log
2. **Debezium connector** lee el WAL → publica evento en topic Kafka
3. **Consumidores:** actualizan MongoDB, notifican microservicios, alimentan Data Warehouse
4. **Latencia:** < 1 segundo, sin impacto en rendimiento transaccional

Flujo B — Event-Driven con Outbox Pattern

Para eventos críticos que requieren atomicidad:

1. Microservicio inicia transacción ACID en PostgreSQL
2. Dentro de la misma transacción: escribe cambio funcional + registro en event_outbox
3. Event relay lee event_outbox y publica en Kafka
4. Si falla la publicación → reintento; si falla la DB → rollback de todo

Catálogo de Eventos Principales

Tabla 33. Eventos principales

Evento	Origen	Destinos	Patrón	Latencia Tolerada
order.created	PostgreSQL	MongoDB, DW, Notificaciones	Outbox	< 2 seg
payment.confirmed	PostgreSQL	MongoDB, DW, Facturación	Outbox	< 1 seg
product.updated	MongoDB	PostgreSQL (stock), DW	CDC reverso	< 5 seg
review.posted	PostgreSQL	MongoDB, DW	CDC	< 3 seg
customer.updated	PostgreSQL	MongoDB, DW	CDC	< 5 seg

Anexos

Diccionario de Datos

Ver archivo separado: docs/Diccionario_Datos.md

Contiene el detalle completo de todas las tablas, columnas, tipos, restricciones y descripciones del módulo transaccional PostgreSQL.

Scripts SQL

DDL (esquema completo) → postgresql/schema/01_ddl_ecommify.sql

Seed data (inserciones de ejemplo) → postgresql/seed_data/01_seed_ecommify.sql

Consultas de Ejemplo

Pedido completo con múltiples ítems → postgresql/queries/01_order_detail.sql

Análisis de ventas por categoría → postgresql/queries/02_sales_by_category.sql